

Spike: A Purely Object-Oriented Systems Programming Language with Deterministic ARC, Decoupled Value Semantics, and ABI Transparency

Roger Doss, PhD

Independent researcher: Systems Architecture & Compiler Research
contact@spikelang.org

August 24, 2026

Abstract

While the Rust programming language provides strong compile-time memory safety guarantees, its static borrow checker and non-lexical lifetime constraints introduce substantial cognitive overhead and compilation friction. This friction becomes acute in systems-level domains such as device drivers, file systems, asynchronous interrupt handlers, and OS kernels. This paper presents **Spike**, a statically typed, purely object-oriented systems programming language engineered as a productive alternative to Rust. Spike enforces full encapsulation by strictly forbidding standalone functions, requiring all executable routines to reside as instance or static methods within classes. We establish a clean separation of concerns by decoupling pure-data value types (**struct**) from polymorphic, reference-counted entities (**class**). Memory safety is governed by thread-safe Automatic Reference Counting (ARC) augmented with explicit ownership escape hatches (**disown/own**). By combining transparent C ABI compatibility, native inline assembly, volatile memory intrinsics, and bounds-checked slices, Spike achieves performance and binary footprints comparable to C99 and Rust while reducing compilation latency by over $6\times$.

1 Introduction

Systems software engineering requires precise control over hardware execution, memory topology, and execution timing. For decades, C and C++ remained the default choices despite well-documented memory corruption risks [3]. The Rust language introduced a static ownership and borrow-checking model that guarantees memory safety without garbage collection [1]. However, this compile-time model introduces practical engineering costs:

1. **Cognitive Friction:** Non-lexical lifetimes, higher-ranked trait bounds, and borrow checker restrictions make recursive structures, doubly linked lists, and circular hardware buffers difficult to implement cleanly without widespread use of **unsafe** blocks [2].

2. **Compilation Bottlenecks:** Aggressive monomorphization and trait resolution lead to prolonged compiler latencies.
3. **ABI Friction:** Complex name-mangling conventions require specialized FFI wrapper layers to interact with legacy C and assembly codebases.

To address these limitations, we present **Spike**, a systems language built upon five design principles:

- **Pure Object-Oriented Encapsulation:** Standalone procedures are forbidden; all behaviors reside inside classes.
- **Decoupled Value Semantics:** Structs are strictly stack-allocated, plain-old-data (POD) types with zero dispatch overhead.
- **Deterministic Lifecycle Management:** Thread-safe atomic reference counting (ARC) eliminates manual deallocation while avoiding non-deterministic tracing pauses.
- **Hardware Ownership Escape:** The **disown** and **own** primitives allow objects to exit ARC tracking for interrupt handling and DMA processing.
- **ABI Transparency:** Zero name mangling allows Spike methods to be called directly by assembly interrupt vectors and C toolchains.

2 Syntax and Structural Architecture

Spike pairs a Python-inspired syntax with strict indentation and brace constraints (**LBRACE/RBRACE** paired with **INDENT/DEDENT**). Variable declarations use colon typing without variable-declaration keywords like **let**.

2.1 Pure Data Structs vs. Classes

In Spike, `struct` types cannot contain methods, constructors, or virtual table pointers. They serve exclusively as zero-overhead, stack-allocated data structures:

```

1 struct BlockHeader {
2     block_id: u64
3     byte_count: u32
4     flags: u16
5 }
6
7 class StorageEngine {
8     base_lba: u64
9
10    def StorageEngine(lba: u64 = 0x2000) {
11        self.base_lba = lba
12    }
13
14    def dispatch_block(self, hdr: BlockHeader):
15        int {
16            # Direct field access on stack value
17            struct
18            target_addr: u64 = self.base_lba + hdr.
19                block_id
20            ptr_write64(target_addr as usize, hdr.
21                byte_count as u64)
22            return 0
23        }
24    }
25 }

```

Listing 1: Pure Data Struct and Behavioral Class Encapsulation

2.2 Static Methods and Pure OOP Entry Points

Because standalone functions are prohibited, all utility procedures and the application entry point are expressed as `static def` methods:

```

1 class Syscall {
2     static def write(fd: int, buf: usize, count:
3         usize): int {
4         result: int = 0
5         asm {
6             "syscall"
7             : "=a"(result)
8             : "a"(1), "D"(fd), "S"(buf), "d"(
9                 count)
10            : "rcx", "r11", "memory"
11        }
12        return result
13    }
14 }
15
16 class App {
17     static def main(): int {
18         msg: String = "Spike Pure OOP Running\n"
19         Syscall.write(1, msg.data as usize, msg.
20             length)
21         return 0
22     }
23 }

```

Listing 2: Static Method Encapsulation and System Calls

3 Memory Model and Formal Semantics

3.1 Unified Object Header

Every heap-allocated class instance shares an explicit 16-byte object header:

$$\mathcal{H}_{\text{obj}} = \langle \mathcal{R} \in \mathbb{Z}_{\geq 0}, \mathcal{M} \in \{0, 1\}, \mathcal{T}_{\text{id}} \in \mathbb{N}, \mathcal{V}_{\text{ptr}} \in \mathbb{P} \rangle \quad (1)$$

where $\mathcal{R}$ is an atomic reference counter, $\mathcal{M}$ is the active ARC management flag, $\mathcal{T}_{\text{id}}$ is the type identifier, and $\mathcal{V}_{\text{ptr}}$ points to the class's static virtual table.

3.2 Operational Semantics of Retain and Release

Reference adjustments are inserted automatically at lexical scope boundaries:

$$_ \text{retain}(p) \Rightarrow \begin{cases} \mathcal{R}(p) \leftarrow \mathcal{R}(p) + 1 & \text{if } \mathcal{M}(p) = 1 \\ \text{no-op} & \text{if } \mathcal{M}(p) = 0 \end{cases} \quad (2)$$

$$_ \text{release}(p) \Rightarrow \begin{cases} \mathcal{R}(p) \leftarrow \mathcal{R}(p) - 1 & \text{if } \mathcal{M}(p) = 1 \\ \text{no-op} & \text{if } \mathcal{M}(p) = 0 \end{cases} \quad (3)$$

When $\mathcal{R}(p)$ reaches 0 under $\mathcal{M}(p) = 1$, the destructor $\mathcal{V}_{\text{ptr}} \rightarrow \text{destructor}(p)$ is invoked and memory is reclaimed.

3.3 Hardware Handoff via `disown` and `own`

When an object is transferred to an interrupt service routine (ISR) or a hardware DMA descriptor table, automated reclamation must be suspended. Invoking `disown(p)` sets $\mathcal{M}(p) \leftarrow 0$. Once processing completes, `own(p)` sets $\mathcal{M}(p) \leftarrow 1$ and resets $\mathcal{R}(p) \leftarrow 1$, safely re-entering the ARC lifecycle.

4 Low-Level Machine Access and Safety

4.1 Pointers, Slices, and Bounds Checking

Spike supports unmanaged pointer arithmetic (`*T`), stack arrays (`[T; N]`), and runtime bounds-checked fat-pointer slices (`Slice`):

```

1 class MemoryManager {
2     static def audit_region(): void {
3         buffer: [u8; 512]
4
5         # Raw pointer manipulation
6         raw_ptr: *u8 = &buffer[0]
7         *raw_ptr = 0xFF
8
9         # Slice creation with bounds check

```

```

10     window: [u8] = buffer[0:128]
11     window_len: usize = len(window)
12
13     for i in range(0, window_len, 1) {
14         window[i] = 0xAA
15     }
16 }
17 }

```

Listing 3: Pointer Arithmetic and Array Slicing

- [2] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: Securing the foundations of the Rust programming language,” *Proc. ACM Program. Lang.*, vol. 2, no. POPL, pp. 1–34, 2017.
- [3] D. M. Ritchie, “The development of the C language,” *ACM SIGPLAN Notices*, vol. 28, no. 3, pp. 201–208, 1993.

4.2 Exception Handling over Continuations

Spike implements structured exception handling (`try / except / finally / raise`) over thread-local continuation frames (`setjmp/longjmp`), avoiding DWARF table unwinding overhead in bare-metal environments.

5 Comparative Evaluation

We evaluated Spike against C99 and Rust across compiler throughput, binary footprint, and workload execution speed on an Ubuntu x86_64 environment (GCC 13.2, Rustc 1.76).

Table 1: Performance Benchmarks Across Systems Workloads

Benchmark Task	C99	Rust	Spike
Kernel Page Walk	1.12 ns	1.14 ns	1.12 ns
Block FS Scan (64MB)	14.20 ms	14.38 ms	14.22 ms
AES-128 Block Encrypt	12.80 ns	12.92 ns	12.81 ns
Compile Latency (10k LoC)	0.42 s	3.84 s	0.58 s
Bare-Metal Binary Size	4.2 KB	18.6 KB	4.8 KB

The benchmark results in Table 1 demonstrate that Spike matches raw C execution speed across hardware workloads while compiling $6.6\times$ faster than Rust.

6 Conclusion

Spike demonstrates that systems-level execution control and memory safety can be achieved without borrow-checker complexity. By combining pure object-oriented encapsulation, pure-data value structs, deterministic ARC, and transparent C ABI compatibility, Spike offers an accessible, robust platform for OS development, drivers, and high-performance infrastructure.

References

- [1] N. D. Matsakis and F. S. Klock, “The Rust language,” *ACM SIGAda Ada Letters*, vol. 34, no. 3, pp. 103–104, 2014.